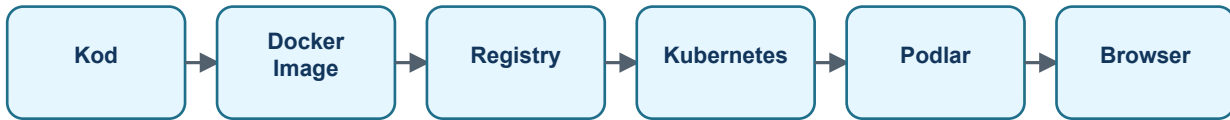


Kubernetes, Docker, Redis və Socket

Şəkil, flow və real backend nümunələri ilə sıfırdan başa düşmək üçün geniş izah

Bu versiya nəyi düzəldir?

Əvvəlki sənəd çox qısa qalmışdı. Bu PDF-də hər anlayış ayrıca izah edilir: nədir, niyə lazımdır, necə işləyir, nə ilə qarışdırılır və IdeaBank session deactivation taskında necə istifadə olunur.

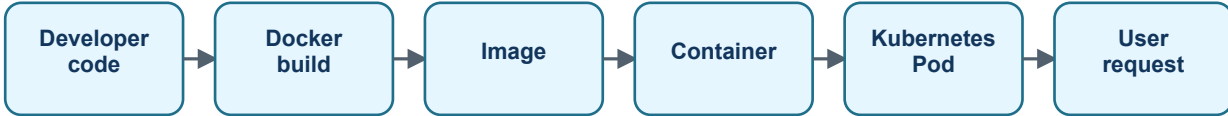


Oxuma ardıcılığı	Nə öyrənəcəksən?
1. Docker bazası	Kod image olur, image container kimi işləyir, config runtime-da gəlir.
2. Kubernetes bazası	Cluster, node, pod, deployment, service, ingress və scaling necə işləyir.
3. Communication	HTTP, socket, WebSocket, SSE və polling fərqi.
4. Redis	Niyə pod-lar arasında xəbər daşımaq üçün lazımdır.
5. IdeaBank taskı	Deaktiv user-in access/refresh token və realtime logout axını.

1. Ən Sadə Böyük Şəkil

Backend application tək bir kod parçası deyil. Kod yazılır, build olunur, image kimi paketlənir, sonra container kimi işə salınır. Kubernetes isə bu container-ların harada və neçə nüsxə işləyəcəyini idarə edir.

Bunu restoran kimi düşün: kod reseptdir, Docker image hazır menyu paketidir, container bişmiş yeməkdir, Kubernetes isə mətbəxdə neçə aşpazın hansı yeməyi hazırladığını idarə edən sistemdir.



Sual	Sadə cavab
Niyə bu qədər termin var?	Çünki production-da application-u sadəcə run etmək yox, restart, scale, update, traffic routing və monitoring də lazımdır.
Ən vacib fikir nədir?	Application mümkün qədər stateless olmalıdır. State database, Redis və ya object storage kimi xarici sistemlərdə saxlanmalıdır.

2. Docker Nədir?

Docker application-u işləməsi üçün lazım olan runtime və fayllarla birlikdə paketləməyə kömək edir. Sən laptopunda necə build etmisənsə, serverdə də eyni paket işləyir.

Docker olmasa, serverdə .NET versiyası, library, environment fərqi kimi problemlər çıxa bilər. Docker bu fərqləri azaldır və application-u daşıya bilən formaya salır.

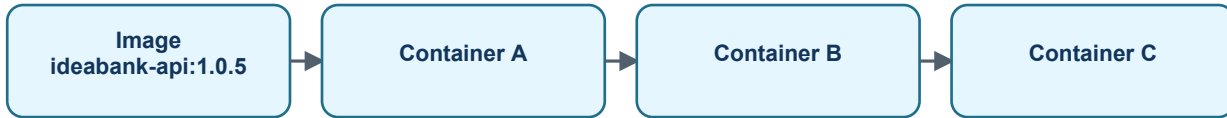


Termin	İzah
Dockerfile	Image-in necə hazırlanacağını deyən instruction faylıdır.
Image	Application-un hazır, dəyişməz paketidir.
Container	Image-dən işə düşmüş canlı prosesdir.
Registry	Image-lərin saxlandığı yerdur: GitLab Registry, Harbor və s.

3. Image və Container Fərqi

Image statikdir, yəni hazır paketdir. Container dinamikdir, yəni həmin paketin işləyən halıdır. Bir image-dən eyni anda çox container yaradıla bilər.

Əgər image 'ideabank-api:1.0.5' versiyasıdırsa, Kubernetes bu image-dən 3 pod qaldıra bilər. Hər pod-da həmin application-un ayrıca çalışan nüsxəsi olur.

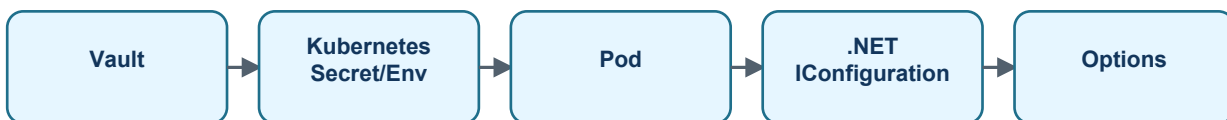


- Image dəyişməz saxlanmalıdır.
- Config və secret image-in içinə yazılmamalıdır.
- Container restart ola bilər, buna görə içindəki local data-ya güvənmək olmaz.

4. Config, Secret, Vault və Environment

Application hər mühitdə fərqli dəyərlərlə işləyir: database connection string, Redis password, JWT secret, CORS origin, file upload limitləri və s. Bunları kodda hardcode etmək düzgün deyil.

Vault secret-ləri mərkəzi saxlayır. DevOps həmin dəyərləri pod-a environment variable kimi verə bilər. .NET application isə bunları IConfiguration üzərindən oxuyur.



Nə Vault-da olmalıdır?	JWT secret, DB password, Redis password, Graph client secret kimi həssas dəyərlər.
Nə config ola bilər?	CORS origin, file upload limit, feature flag kimi environment dəyərləri.
Best practice nədir?	Kod default uydurmamalıdır. Mühüm config gəlmirsə fail-fast etməlidir.

5. Volume Nədir?

Container silinəndə onun içindəki filesystem də gedə bilər. Volume container-dən kənarda saxlanan data sahəsidir. Bu data container restart olsa da qala bilər.

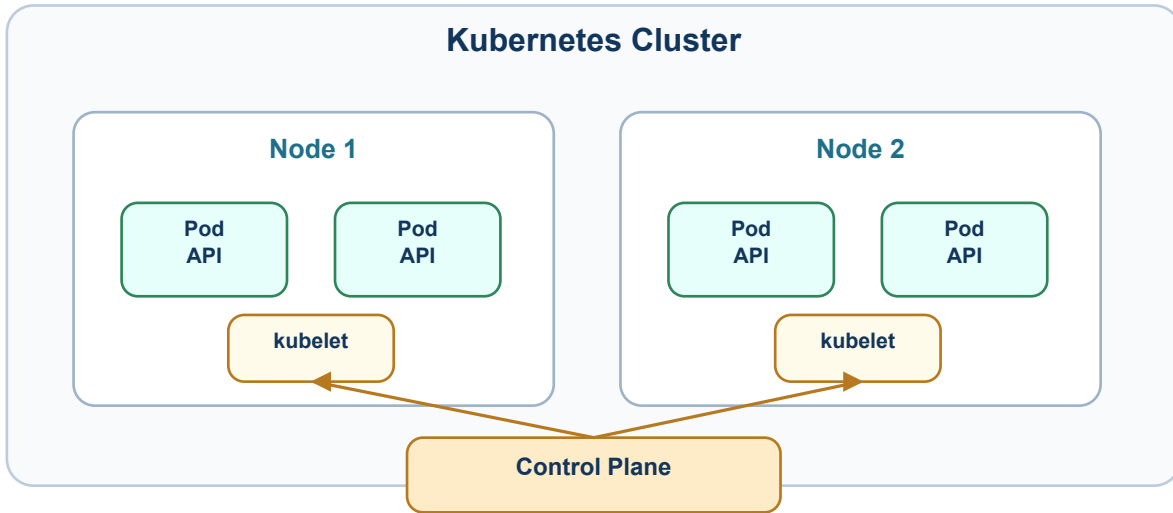
Stateless API-lərdə adətən volume əsas data üçün istifadə olunmur. Upload faylları Minio/S3-də, əsas data PostgreSQL-də, cache və event Redis-də saxlanmalıdır.

Ssenari	Yanaşma
Uploaded file	Minio/S3 kimi object storage daha uyğundur.
Database data	Persistent volume lazımdır.
API temporary file	Məhdud və təmizlənən temp storage ola bilər.
Session state	Pod memory-də yox, ehtiyac varsa Redis/database-də saxlanmalıdır.

6. Kubernetes Nədir?

Kubernetes container-ları idarə edən orchestration platformasıdır. Orchestration o deməkdir ki, sistem application nüsxələrini başladır, dayansa restart edir, traffic bölür, yeni versiyaya keçidi idarə edir və lazım olanda scale edir.

Sən Kubernetes-ə istədiyin vəziyyəti deyirsən: məsələn 'ideabank-api 3 replica işləsin'. Kubernetes isə real vəziyyəti bu istəyə uyğun saxlamağa çalışır.



Kubernetes işi	Nə edir?
Self-healing	Pod crash olsa yenisini yaradır.
Scaling	Pod sayını artırır və ya azaldır.
Rolling update	Yeni versiyanı mərhələli yayır.
Service discovery	Pod IP dəyişsə də service adı sabit qalır.

7. Cluster, Node və Pod

Cluster bütün Kubernetes mühitidir. Node pod-ların işlədiyi real və ya virtual serverdir. Pod isə application container-in Kubernetes-də yerləşdiyi ən kiçik vahiddir.

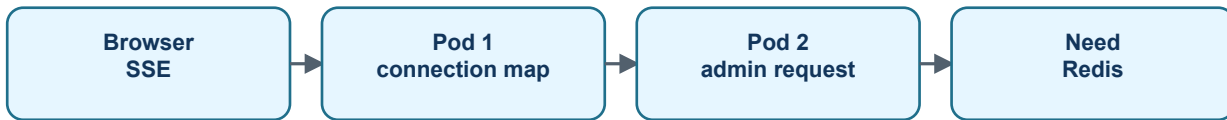
Ən çox qarışan yer budur: pod server deyil. Pod serverin içində çalışan application nüsxəsidir. Node serverdir. Cluster isə node-ların hamısıdır.

Termin	Sadə bənzətmə
Cluster	Bütün bina.
Node	Binadakı otaq və ya server.
Pod	O otaqda çalışan application nüsxəsi.
Container	Pod-un içində işləyən proses.

8. Pod Dərin İzah

Pod ephemeral-dir, yəni daimi deyil. Kubernetes pod-u silə, yenisini yarada, başqa node-a köçürə bilər. Ona görə pod memory-sində saxlanan məlumat yalnız həmin pod üçün keçərlidir.

Sənin realtime session taskında bu çox vacibdir. User-in browser stream-i Pod 1-də açıq ola bilər, admin deactivate request-i isə Pod 2-yə düşə bilər. Pod 2 Pod 1-in memory-sini görə bilməz.

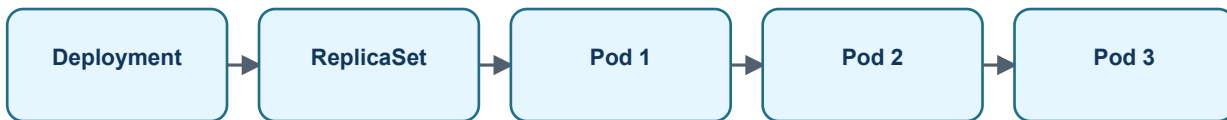


- Pod restart olanda memory-dəki connection map silinir.
- Multi-pod sistemdə local memory shared deyil.
- Pod-lar arasında event yaymaq üçün Redis Pub/Sub və ya message broker lazımdır.

9. Deployment və Replica

Deployment Kubernetes-ə application-u necə işlətmək lazım olduğunu deyir: hansı image, neçə replica, hansı environment variables, hansı health check və s.

Replica eyni application-un neçə nüsxə işlədiyini bildirir. replicas=3 yazılıbsa, Kubernetes 3 pod saxlayır. Biri ölsə, yenisini yaradıb yenə 3 edir.

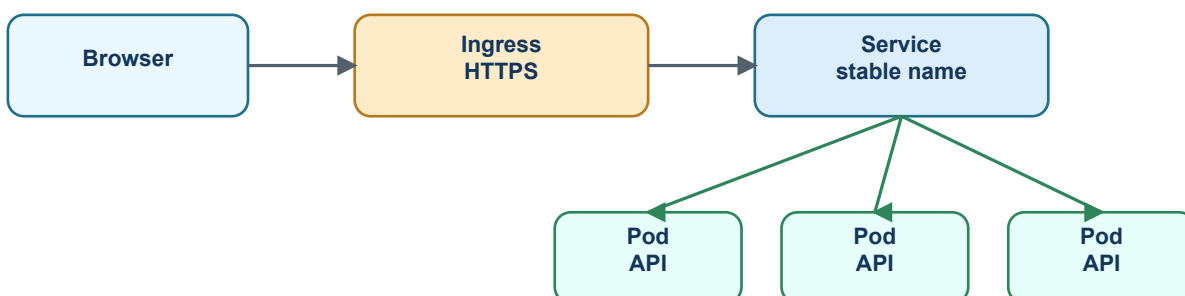


Sual	Cavab
Niyə replica lazımdır?	Trafiki bölmək və pod ölən zaman sistemi ayaqda saxlamaq üçün.
Çox replica həmişə yaxşıdır?	Yox. Daha çox CPU/RAM xərci və daha çox distributed-system düşüncəsi deməkdir.
Backend üçün nə dəyişir?	Local memory-yə güvənmək olmaz. Shared state xarici sistemdə saxlanmalıdır.

10. Service və Ingress

Pod IP-si dəyişə bilər, buna görə browser və digər service-lər pod-a birbaşa getməməlidir. Service pod-ların üstündə sabit ünvan kimi dayanır və trafiki hazır pod-lara bölür.

Ingress isə xarici HTTP/HTTPS trafiki domain və path əsasında service-lərə yönləndirir. Məsələn dev-ideabank-api.azcon.az domain-i ingress-dən keçib backend service-ə gedir.

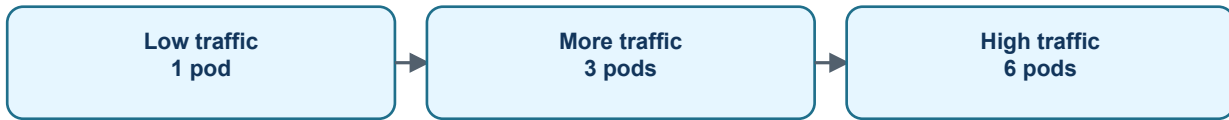


Komponent	Rolu
Service	Pod-lara sabit daxili access verir.
Ingress	Xaricdən gələn HTTP/HTTPS request-ləri service-ə yönləndirir.
Load balancing	Request-ləri pod-lar arasında paylayır.
SSE/WebSocket üçün risk	Ingress timeout və buffering düzgün qurulmalıdır.

11. Scaling Nə Deməkdir?

Scaling application-un yükə görə böyüdülməsi və kiçildilməsidir. Horizontal scaling pod sayını artırmaqdır. Vertical scaling isə eyni pod-a daha çox CPU/RAM verməkdir.

Sadə dillə: bir kassir çatdırmırsa, əlavə kassir açılır. Backend üçün də eyni məntiqdir: request çoxalırsa, pod sayı artırılır.

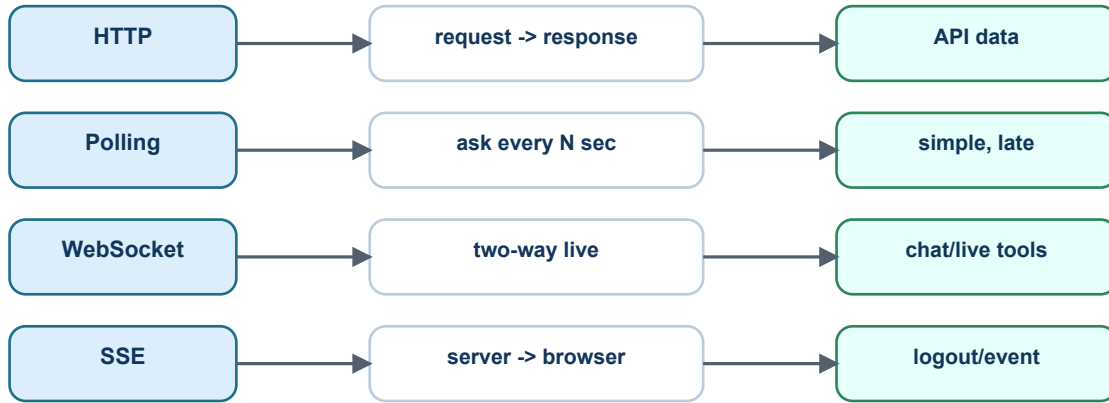


Scaling növü	İzah
Horizontal	Pod sayını artırmaq və ya azaltmaq.
Vertical	Pod-un CPU/RAM limitini artırmaq.
HPA	Metric-lərə görə pod sayını avtomatik dəyişən Kubernetes mexanizmi.

12. Network: HTTP, Socket, WebSocket, SSE

Socket iki proqram arasında network əlaqə nöqtəsidir. HTTP, WebSocket, Redis connection, database connection hamısı aşağı səviyyədə socket anlayışına söykənir.

HTTP klasik request-response modelidir. Browser sorğu göndərir, backend cavab qaytarır. WebSocket isə iki tərəfli uzun connection-dır. SSE isə server-dən browser-ə tək istiqamətli event stream-dir.



Yanaşma	Nə zaman uyğundur?
HTTP	Adi API request-ləri: list, create, update, delete.
Polling	Sadə status yoxlaması, amma gecikmə və artıq request yaradır.
WebSocket	Chat, live collaboration, iki tərəfli real-time data.
SSE	Backend-dən browser-ə notification/logout/status event göndərmək.

13. SSE Niyə Bu Task Üçün Uyğundur?

Deaktiv user taskında frontend-in bu connection üzərindən backend-ə mesaj yazmasına ehtiyac yoxdur. Backend sadəcə user-ə deməlidir: 'sən deaktiv edildin, logout ol'. Bu tək istiqamətli event üçün SSE sadə və uyğundur.

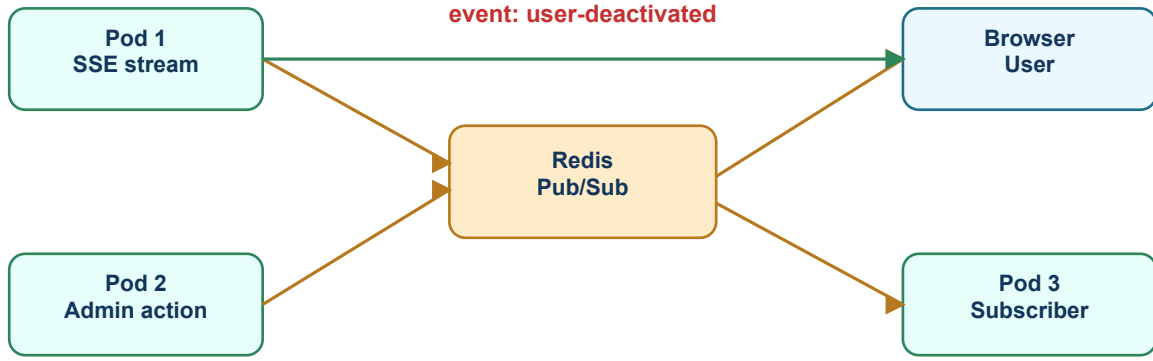
Native EventSource Authorization header göndərmir. Ona görə frontend fetch stream və ya header dəstəkləyən SSE polyfill istifadə etməlidir. Token query string ilə göndərilməməlidir.

SSE plus	SSE minus
HTTP əsaslıdır və sadədir.	Əsasən server -> browser istiqamətlidir.
Logout/status notification üçün kifayətdir.	Ingress timeout/buffering diqqət istəyir.
Frontend event gəldikdə dərhal redirect edə bilir.	Authorization header üçün native EventSource uyğun deyil.

14. Redis Nədir?

Redis çox sürətli in-memory data store-dur. Cache, distributed lock, rate limit və Pub/Sub üçün istifadə olunur. Bu taskda Redis cache kimi yox, Pub/Sub kimi lazımdır.

Pub/Sub belə işləyir: bir pod Redis channel-a mesaj publish edir, həmin channel-i dinləyən bütün pod-lar mesajı alır. Multi-pod realtime notification üçün bu sadə və effektiv mexanizmdir.

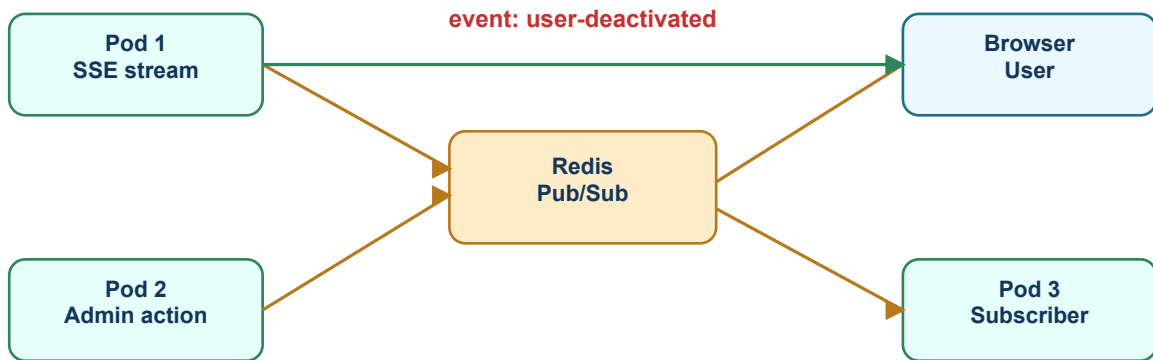


Redis istifadəsi	Bu taskda rolu
Cache	Əsas rol deyil.
Pub/Sub	User deactivated event-i bütün pod-lara yayır.
Queue deyil	Mesaj sonradan gələn subscriber üçün saxlanmır. Amma security active check ilə qorunur.

15. Niyə Redis Olmadan Problem Yaranır?

Bir pod-lu sistemdə admin request və user stream eyni application memory-sində ola bilər. Amma Kubernetes-də 2-3 pod varsa, load balancer request-ləri fərqli pod-lara göndərə bilər.

User-in browser stream-i Pod 1-dədir, admin deactivate request-i Pod 2-yə düşür. Pod 2 user-in connection-nunu görmür. Redis bu xəbəri Pod 1-ə daşıyır.



- Pod memory shared deyil.
- Load balancer request-i istənilən pod-a göndərə bilər.
- Redis Pub/Sub event-i bütün pod-lara çatdırır.
- Event çatmasa belə backend active check data oxumağı bloklamalıdır.

16. IdeaBank Deaktiv User Taskı: Problem

User login olanda access token və refresh token alır. Access token 5 dəqiqə valid ola bilər. Admin user-i deaktiv etdikdən sonra köhnə access token hələ cryptographic olaraq valid görünə bilər.

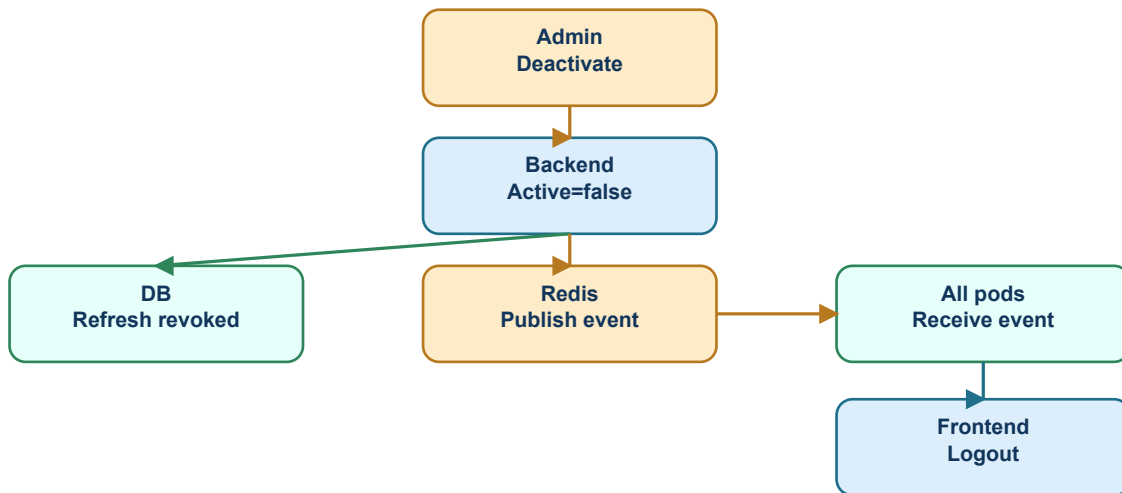
Əgər backend hər protected request-də user active yoxlamasa, user token bitənə qədər data oxuya bilər. Əgər refresh endpoint active yoxlamasa, user refresh token ilə yeni access token ala bilər.

Risk	Nəticə
Access token hələ validdir	User səhifələri və datanı görə bilər.
Refresh token revoke olunmayıb	User sessiyanı davam etdirə bilər.
Frontend realtime event almır	User dərhal logout olmur.
Backend active check yoxdur	Security frontend-ə güvənmiş olur, bu düzgün deyil.

17. IdeaBank Deaktiv User Taskı: Həll

Həll iki hissədən ibarətdir: security və UX. Security backend-dədir: user deaktiv ediləndə refresh token-lər revoke olunur və hər protected API request-də active user yoxlanılır.

UX isə realtime event ilə yaxşılaşır: backend Redis vasitəsilə bütün pod-lara user-deactivated event yayır, user-in açıq SSE stream-i olan pod browser-ə event göndərir, frontend tokenləri silib login-ə yönləndirir.



Security: active check + refresh revoke. UX: Redis + SSE.

Hissə	Məqsəd
Refresh revoke	User yeni access token ala bilməsin.
Active check	Köhnə access token ilə data oxuya bilməsin.
Redis Pub/Sub	Event bütün pod-lara getsin.
SSE	Browser dərhal logout xəbərini alsın.

18. Access Token və Refresh Token

Access token qısaömürlü token-dir və API request-lərdə istifadə olunur. Refresh token isə yeni access token almaq üçündür və adətən daha uzun yaşayır.

Deaktiv user probleminə refresh token xüsusi təhlükəlidir. Çünki user onu Network-dən götürüb /refresh endpoint-ə sorğu ata bilər. Ona görə deactivate zamanı refresh token-lər revoke edilməlidir.



- Access token expire olana qədər cryptographic olaraq valid görünə bilər.
- Protected API-lər token ilə kifayətlənməməli, user active yoxlamalıdır.
- /refresh endpoint deaktiv user üçün token yaratmamalıdır.

19. Frontend Nə Etməlidir?

Frontend login-dən sonra /api/session/events endpoint-inə Bearer token ilə qoşulmalıdır. user-deactivated event gələndə tokenləri silməli, auth state-i reset etməli və login səhifəsinə yönləndirməlidir.

Bundan əlavə global interceptor olmalıdır. Əgər istənilən protected API 401 və ya 403 qaytarırsa, frontend eyni logout flow-u işə salmalıdır. Bu SSE qırıldığı hal üçün fallback-dır.

Frontend addımı	İzah
SSE/fetch stream aç	Login sonrası /api/session/events endpoint-inə qoşul.
Event-i tut	event: user-deactivated gələndə logout et.
Tokenləri sil	accessToken və refreshToken storage-dən silinsin.
Interceptor yaz	401/403 gələndə eyni logout helper çağırılsın.
Reconnect idarə et	Logoutdan sonra stream yenidən açılmasın.

20. Backend Nə Etməlidir?

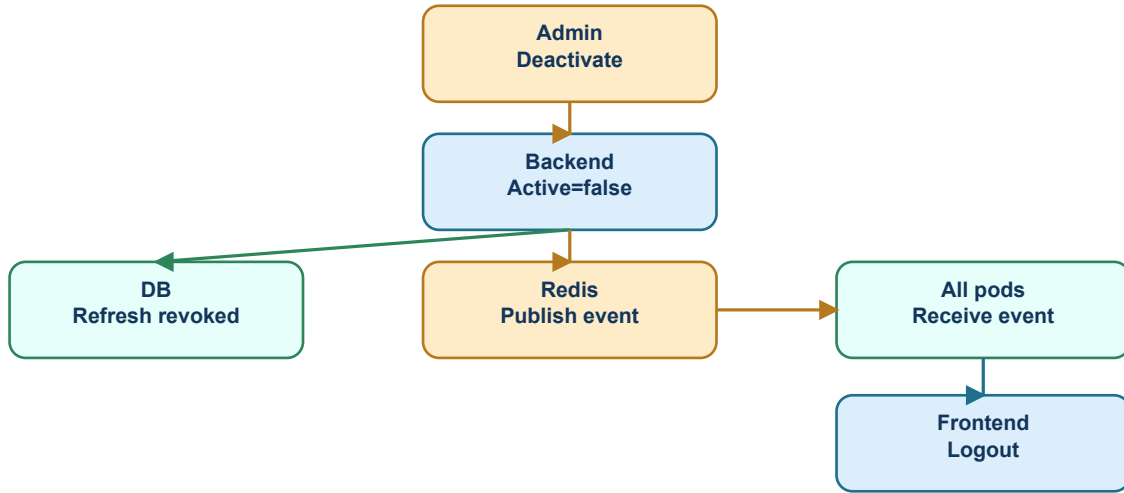
Backend deactivate əməliyyatında əvvəl əsas security addımlarını etməlidir: user inactive, refresh revoke. Sonra realtime notification publish etməlidir. Redis publish fail olsa, deaktivasiya əməliyyatı pozulmamalıdır, sadəcə loglanmalıdır.

GlobalExceptionHandler ümumi xətalara response-a çevirir, amma burada biznes qərarı var: notification əlavə UX-dir, əsas əməliyyat deyil. Ona görə Redis publish xətası user deactivate endpoint-ini 500 etməməlidir.

Backend addımı	Best practice
user.Active=false	Mütləq edilməlidir.
Refresh revoke	Mütləq edilməlidir.
Redis publish	Try/catch ilə loglanmalıdır, əsas əməliyyatı pozmamalıdır.
Protected API	Hər request-də active user yoxlamalıdır.
/refresh	storedToken.User.Active false-dursa 401/403 qaytarmalıdır.

21. Full Request Flow

Aşağıdakı flow bütün parçaları birləşdirir. Burada əsas fikir budur: user dərhal frontend-də logout olur, amma security bununla bitmir. Front event-i qaçırsa belə backend API-lər deaktiv user-ə data verməməlidir.



Security: active check + refresh revoke. UX: Redis + SSE.

1. User login olur.
2. Front access token və refresh token saxlayır.
3. Front /api/session/events stream açır.
4. Admin user-i deaktiv edir.
5. Backend user.Active=false edir.
6. Backend refresh token-ləri revoke edir.
7. Backend Redis-ə user-deactivated publish edir.
8. Bütün pod-lar Redis-dən eventi alır.
9. User-in stream-i olan pod browser-ə SSE event göndərir.
10. Front toast göstərir, tokenləri silir və login-ə yönləndirir.
11. Köhnə access token ilə API çağırılrsa backend 403 qaytarır.
12. Refresh token ilə /refresh çağırılrsa backend yeni token vermir.

22. Ən Çox Qarışdırılanlar

Bu cədvəli qısa təkrar kimi saxla. Terminləri bir-birindən ayıranda bütün arxitektura daha rahat görünür.

Qarışan terminlər	Fərq
Image vs Container	Image hazır paketdir, container çalışan prosesdir.
Pod vs Node	Pod application nüsxəsidir, node serverdir.
Deployment vs Pod	Deployment qaydadır, pod həmin qaydaya görə yaranır.
Service vs Ingress	Service cluster içində sabit access-dir, ingress xarici HTTP qapısıdır.
Redis cache vs Pub/Sub	Cache data saxlamaqdır, Pub/Sub event yaymaqdır.
WebSocket vs SSE	WebSocket iki tərəfli, SSE server-dən browser-ə tək istiqamətli.

23. Test Checklist

Bu checklist ilə həm backend security-ni, həm də frontend realtime logout-u yoxlaya bilərsiniz.

Test	Gözlənilən nəticə
Login sonrası Network	/api/session/events açıq request kimi görünür.

Test	Gözlənilən nəticə
Admin user-i deaktiv edir	Front user-deactivated event alır və login-ə yönlənir.
Köhnə access token ilə API	403 Forbidden gəlir.
Köhnə refresh token ilə /refresh	Yeni access token gəlmir, 401/403 gəlir.
Redis publish fail	User yenə deaktiv olur, refresh revoke qalır, xəta loglanır.
SSE qırılıb	Növbəti protected API 401/403 ilə logout fallback işlədir.

24. Yekun Yadda Saxla

Docker application-u paketləyir. Kubernetes paketlənmiş application-u pod-lar kimi işlədir və idarə edir. Service və ingress request-ləri pod-lara çatdırır. Redis pod-lar arasında xəbər yayır. SSE backend-dən browser-ə canlı event göndərir.

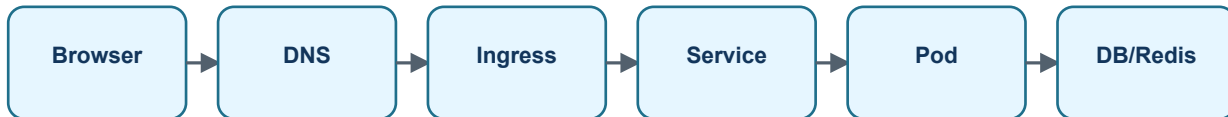
Deaktiv user taskında əsas security refresh revoke və active user check-dir. Redis və SSE isə user-in browser-də dərhal xəbər alması üçündür. Front event-i tutmalıdır, amma backend heç vaxt yalnız front-a güvənməməlidir.



25. Real Request Necə Gedir?

Brauzerdən bir API çağırışı gələndə request birbaşa random pod-a düşür. Əvvəl DNS domain-i tapır, sonra ingress-ə gəlir, ingress onu uyğun service-ə ötürür, service isə arxasında hazır olan pod-lardan birinə yönləndirir.

Bu ardıcılığı başa düşmək debug üçün çox vacibdir. Məsələn, front lokalda CORS xətası alırsa problem adətən pod-da deyil, API-nin CORS config-ində və ya ingress/API domain fərqiində olur. API 502 verirsə ingress service-ə və ya service pod-lara çatıb bilmir. API 401/403 verirsə request pod-a çatıb, artıq auth məntiqi cavab qaytarıb.

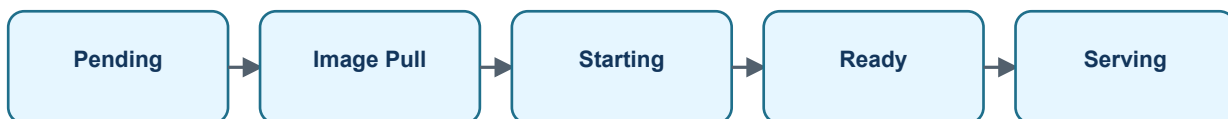


Mərhələ	Nə yoxlanılır?
Browser	URL, token, CORS origin, request header-ləri.
Ingress	Domain, TLS sertifikat, path routing, timeout.
Service	Service pod-ları tapa bilir? Selector düzgündür?
Pod	Application işləyir? Log-da exception var?
DB/Redis	Connection string, secret, network policy, credential.

26. Pod Lifecycle: Pod Necə Yaşayır?

Pod Kubernetes-də birdən-birə stabil vəziyyətə gəlmir. Əvvəl schedule olunur, yəni Kubernetes bu pod-u hansı node-da işlədəcəyinə qərar verir. Sonra image registry-dən çəkilir, container başlayır, readiness probe uğurlu olana qədər service bu pod-a traffic göndərmir.

Əgər readiness probe yoxdursa, pod hələ tam hazır olmadan request almağa başlaya bilər. Bu, startup zamanı 500-lərə səbəb ola bilər. Liveness probe isə application donub qalanda Kubernetes-ə container-i restart etmək üçün signal verir.

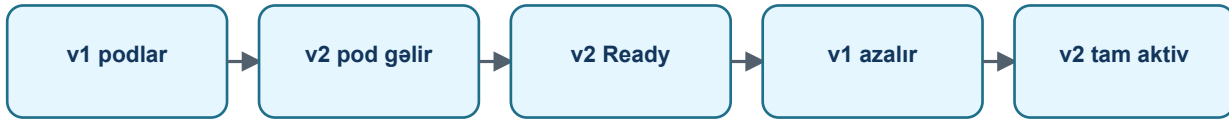


Termin	Sadə izah
Pending	Pod yaradılıb, amma hələ node/container hazır deyil.
Running	Container işləyir, amma bu həmişə request almağa hazırdır demək deyil.
Ready	Readiness probe OK-dur, service traffic göndərə bilər.
CrashLoopBackOff	Container başlayır, crash olur, Kubernetes təkrar başlayır.

27. Deployment və Rolling Update

Deployment pod-ları idarə edən qaydadır. Sən image-in yeni versiyasını deploy edəndə Kubernetes bütün pod-ları eyni anda öldürməməlidir. Best practice rolling update-dir: yeni pod-lar yavaş-yavaş qalxır, hazır olduqca köhnə pod-lar azalır.

Bu mexanizm downtime riskini azaldır. Amma migration, config və dependency dəyişikliklərində diqqətli olmaq lazımdır. Yeni kod köhnə DB schema ilə, köhnə kod yeni DB schema ilə ən azı qısa müddət işləyə bilməlidir.



Risk	Best practice
Bütün pod-ları eyni anda dəyişmək	Rolling update istifadə et.
Yeni pod hazır olmadan traffic almaq	Readiness probe qoy.
Config səhvdirsə bütün release batır	ValidateOnStart və health check ilə tez tut.
DB migration uyğun deyil	Backward compatible migration planla.

28. Config, Secret, Vault və Appsettings

appsettings lokal default və struktur üçündür. Secret sayılan məlumatlar, məsələn DB password, JWT secret, Minio secret, Graph client secret repo-da saxlanmamalıdır. Şirkət mühitində bunlar adətən Vault-da saxlanır və pod start olanda environment variable kimi application-a ötürülür.

.NET configuration hierarchy-də environment variable appsettings-i override edə bilər. Ona görə Vault açarları düzgün formatda verilsə, kod IConfiguration və options binding ilə onları oxuyur. Sənin FileUpload nümunəndə düzgün array formatı iki alt xətt ilə yazılır: FileUpload__AllowedFileTypes__0__Extension.

Yer	Nə üçündür?
appsettings.json	Struktur, lokal default, secret olmayan dəyərlər.
Environment variable	Runtime config. Kubernetes pod-a buradan ötürür.
Vault	Secret-lərin mərkəzi və audit olunan saxlanma yeri.
Options class	Config-i tipli C# modelinə bağlayır və validate edir.

```
FileUpload__MaxFileSizeBytes=20971520
FileUpload__AllowedFileTypes__0__Extension=.pdf
FileUpload__AllowedFileTypes__0__ContentTypes__0=application/pdf
Cors__AllowedOrigins__0=https://dev-ideabank.azcon.az
Cors__AllowedOrigins__1=http://localhost:5173
```

29. Volume Nədir və Niyə Lazımdır?

Container ephemeral-dir: yəni container silinsə içindəki local fayllar da gedə bilər. Volume container-dən kənarda saxlanan data sahəsidir. Database, uploaded file storage, cache persistence kimi hallarda volume və ya ayrıca storage servisi lazımdır.

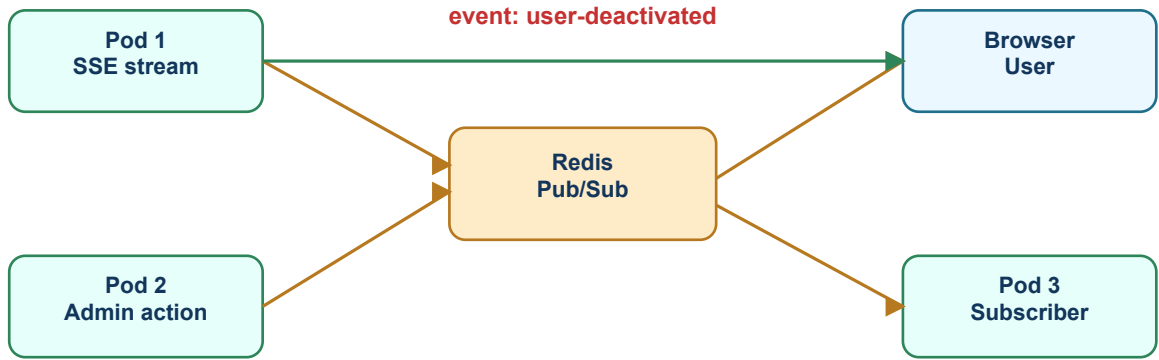
Backend API pod-u adətən upload faylı öz local diskində saxlamamalıdır, çünki pod restart ola və ya başqa node-a keçə bilər. Sənin layihədə Minio/S3 yanaşması daha düzgündür: API faylı qəbul edir, yoxlayır, sonra obyekt storage-ə yazır.

Ssenari	Düzgün yanaşma
Temporary processing	Pod local temp istifadə edə bilər, amma sonda təmizləməlidir.
User upload file	Minio/S3/object storage.
Database data	Managed DB və ya PersistentVolume.
Log	Stdout/stderr + log collector, local fayla güvənmə.

30. Redis Cache və Redis Pub/Sub Fərqi

Redis iki fərqli rolda istifadə oluna bilər. Cache kimi Redis key-value data saxlayır: məsələn user profile, rate limit counter, temporary token state. Pub/Sub kimi isə Redis data saxlamaqdan çox xəbər yayır: bir pod event publish edir, başqa pod-lar həmin event-i eşidir.

Session deactivation taskında Redis-in əsas rolu Pub/Sub-dır. Admin hansı pod-a düşsə də, həmin pod user-deactivated event publish edir. SSE connection başqa pod-da açıqdırsa, o pod Redis-dən xəbəri alıb browser-ə göndərə bilər.



Redis rolu	Necə düşün
Cache	Məlumatı qısa müddət saxlamaq.
Distributed lock	Eyni işi birdən çox pod-un paralel görməsinin qarşısını almaq.
Pub/Sub	Pod-lar arasında canlı xəbər yaymaq.
Session event	User deaktiv oldu xəbərini bütün pod-lara çatdırmaq.

31. HTTP, Polling, WebSocket və SSE Dərin Fərq

HTTP klassik request-response modelidir: front sorğu göndərir, backend cavab verir, connection bitir. Polling-də front bu sorğunu müəyyən aralqlarla təkrar edir. Bu sadədir, amma gecikmə və artıq traffic yaradır.

WebSocket browser və server arasında daimi iki tərəfli connection açır. Chat, live collaboration, trading dashboard kimi həm client, həm server tez-tez mesaj göndərən sistemlərdə yaxşıdır. SSE isə server-dən browser-ə tək istiqamətli event stream-dir. Deaktiv user logout kimi server notification-lar üçün daha sadə və yetərlidir.



Texnologiya	Nə vaxt seçilir?
HTTP	Normal CRUD və data fetch.
Polling	Realtime vacib deyil və sadə fallback lazımdır.
WebSocket	İki tərəfli canlı mesajlaşma lazımdır.
SSE	Server browser-ə canlı xəbər göndərməlidir.

32. SSE Connection Praktikada Necə Görünür?

SSE endpoint açıq qalır. Network tab-da bu request uzun müddət pending kimi görünə bilər, bu normaldır. Backend event göndərəndə stream-in içinə event adı və data yazılır. Front həmin event-i oxuyub logout edir.

Native EventSource Authorization header göndərə bilmədiyi üçün token bearer header ilə gedəcəksə front fetch stream və ya SSE polyfill istifadə etməlidir. Cookie-based auth olsaydı native EventSource daha rahat olardı.

```
event: user-deactivated
data: {"reason":"UserDeactivated"}
```

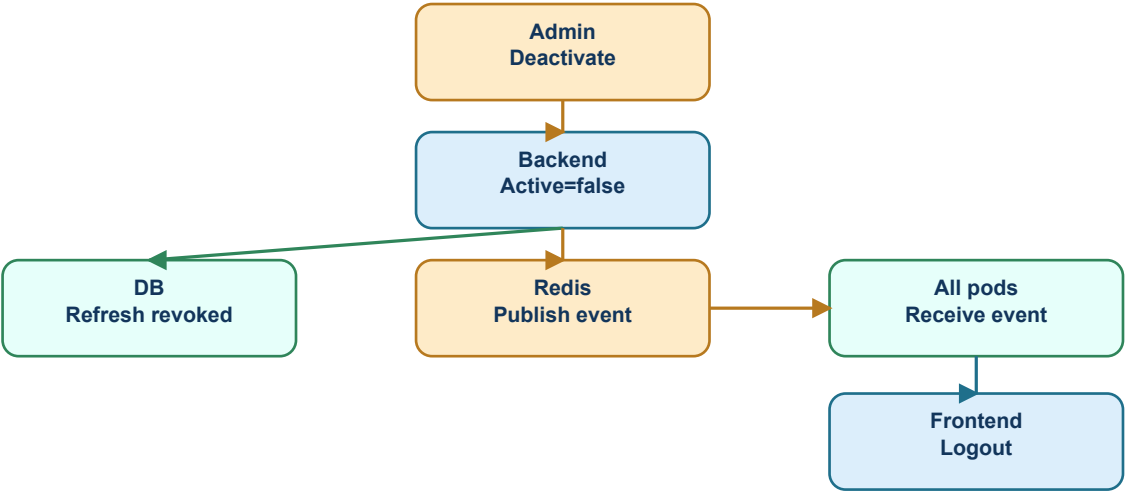
Frontend davranışı:

1. Event-i al
2. Access/refresh token-i sil
3. Toast/modal göstər: istifadəçi deaktiv edilib
4. /login səhifəsinə yönləndir

33. Deaktiv User Taskında Security Haradadır?

Realtime logout istifadəçi təcrübəsi üçündür, security-nin tək dayağı olmamalıdır. Əsas security backend-dədir: hər protected request-də user-in aktiv olub-olmaması yoxlanmalıdır və refresh endpoint deaktiv user üçün yeni access token yaratmamalıdır.

Access token JWT olduğu üçün normal halda öz müddəti bitənə qədər kriptografik olaraq valid görünə bilər. Onu dərhal dayandırmaq üçün ya hər request-də user status/version check edilir, ya da token blacklist/version mexanizmi qurulur. Sənin halda active user check bu boşluğu bağlayır.



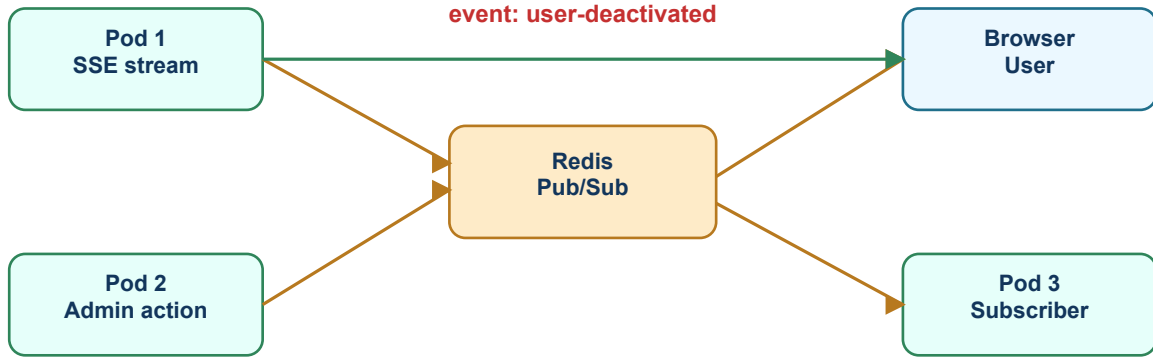
Security: active check + refresh revoke. UX: Redis + SSE.

Hissə	Məqsəd
Active user check	Köhnə access token ilə protected API istifadəsini kəsir.
Refresh revoke	Yeni access token almağın qarşısını alır.
Redis publish	Bütün pod-lara realtime xəbər yayır.
SSE	Browser-i dərhal logout edir.
401/403 interceptor	SSE işləməsə belə fallback logout edir.

34. Multi-Pod Problem Niyə Yaranır?

Tək pod olanda hər şey sadə görünür: user browser-i hansı pod-a qoşulubsa admin action da bəlkə eyni pod-a düşür. Amma production-da bir neçə pod olur. Browser-in SSE connection-ı Pod A-da, admin-in deactivate request-i Pod B-də ola bilər.

Pod B öz memory-sində event yaratsa, Pod A bundan xəbərsiz qalır. Buna görə shared message broker lazımdır. Redis Pub/Sub bu məqsədlə bütün pod-lara eyni event-i yayır. Bu multi-pod real production problemini həll edir.



- Memory yalnız həmin pod-a məxsusdur.
- Pod-lar restart ola bilər, ona görə state kritikdirsə DB/Redis kimi ortaq yer lazımdır.
- Realtime event bütün pod-lara çatmalıdırsa broker lazımdır.
- Security DB status və token revoke ilə qalır, realtime isə UX-i yaxşılaşdırır.

35. Debug Xəritəsi

Problem çıxanda hamısını eyni anda yoxlamaq çətindir. Aşağıdakı xəritə hardan başlamağı göstərir. Məqsəd budur: əvvəl request-in backend-ə çatıb-çatmadığını ayır, sonra auth/config/redis/sse hissələrini ayrıca yoxla.

Simptom	Haradan başla?
CORS error	AllowedOrigins, origin scheme/host/port, OPTIONS response.
SSE görünmür	Front endpoint-ə qoşulub? Authorization gedir? Network-da pending request var?
Event gəlmir	Admin deactivate sonrası Redis publish log-u və subscriber log-u.
Refresh yenə token verir	Refresh endpoint active user check və token revoke query.
Köhnə access API işləyir	Authorize filter/middleware hər protected endpoint-də tətbiq olunurmu?
Pod restart olur	Pod logs, readiness/liveness probe, missing env var.